

Installation Guide



WARESAFE

IoT-based Security System with Cybersecurity Simulation

Game Development | Cybersecurity | Internet of Things

Joy Adelia Sihombing

001202300077

Zafyra Raisidha Azhari

001202300032

Zelika Hannah

001202300067

TABLE OF CONTENTS

<i>WareSafe Product</i>	<i>01</i>
<i>Product Requirements</i>	<i>02</i>
<i>Software Installation</i>	<i>03</i>
<i>IoT System Development</i>	<i>05</i>
<i>Dashboard Monitoring Development</i>	<i>07</i>
<i>Cyber Attack Injection</i>	<i>09</i>
<i>Game Simulation Development</i>	<i>10</i>

Welcome to WARESAFE !



WARESAFE

IoT-based Security System with Cybersecurity Simulation

Waresafe is an integrated solution for smart factory and warehouse security. This platform serves a dual purpose: it provides users with real-time monitoring of the physical workspace while offering an interactive cybersecurity training environment.

WARESAFE FEATURES



**Smart
Sensor
System**

Through our centralized dashboard, users can track live sensor data from secured areas, including RFID-based access control, motion detection in offices, and vibration monitoring for critical assets. By presenting all monitoring information in one place, the dashboard helps users observe system activity and respond to security events more efficiently.



**Cyber Attack
Simulation
Scenario**

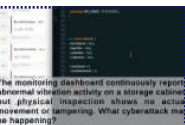
To help users better recognize and respond to modern digital threats, WareSafe includes a cybersecurity learning module that provides a safe environment for practicing cyber-attack scenarios. Users can explore interactive simulations and complete quiz-based learning activities without affecting the actual monitoring system. WareSafe is designed to keep both your physical and digital workplace safe!



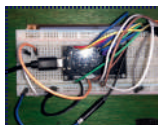
**Game
Simulation
for Education**



**Real-Time
Dashboard
Monitoring**



The monitoring dashboard continuously reports abnormal vibration activity on a storage cabinet but physical inspection shows no actual movement or tampering. What cyberattack may be happening?





Software Requirements

- ✔ Visual Studio Code
- ✔ Arduino IDE >= 2.3.2
- ✔ MQTTX v1.13.0
- ✔ XAMPP MySQL v3.3.0
- ✔ Laravel Framework 12.62.0
- ✔ PHP >= 8.2.12
- ✔ React.js Frontend
- ✔ Unity Hub

Note: Download the latest stable version that is compatible with your operating system

Hardware Requirements

- ✔ ESP32 Microcontroller
- ✔ Jumper Wires
- ✔ Power Supply
- ✔ Breadboard
- ✔ USB Cable

Office Area

- ✔ IR Infrared Sensor
- ✔ Door Reed Switch
- ✔ Vibration Sensor
- ✔ Buzzer

Warehouse Area

- ✔ RFID RC522
- ✔ Door Reed Switch
- ✔ LCD I2C
- ✔ LED Light Indicators
- ✔ Buzzer



Arduino IDE

Note:

- Download the latest stable version
- Ensure that the version meets the minimum system requirement.

Install Arduino IDE

- Go to the official Arduino website.
- Click the Download button.



Setup Arduino IDE

- Connect Arduino to Computer. Use a USB cable.
- Open Arduino IDE. Run the newly installed Arduino IDE software.

Select Board and Port

- Select Board: Click "Tools" → "Board", then select the appropriate board type
- Select Port: Click "Tools" → "Port", then select the port connected to the ESP32 board.



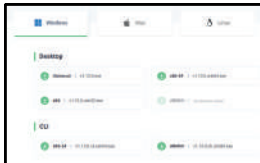
Upload Program to Arduino

- Code the program.
- Click the "Upload" button (right arrow icon) to upload the code to the ESP32 board.

MQTTX

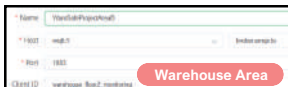
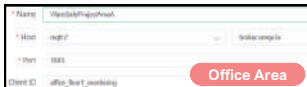
Install MQTTX

- Go to the MQTTX Official Download Page.
- Click the Download button.
- Run the installer (.exe for Windows or .dmg for macOS).



Read Arduino Messages

- Open your MQTTX desktop application.
- Create a new connection using the same broker address.





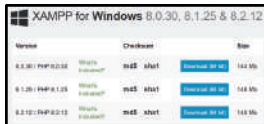
XAMPP

Note:

- Download the latest stable version
- Ensure that the version meets the minimum system requirement.

Install XAMPP

- Go to the official XAMPP website.
- Execute the installer and complete the installation process.
- Open XAMPP Control Panel and start Apache and MySQL services.



Download Composer

- Download the composer.
- Follow the installation wizard and complete the setup.
- Verify Installation: Open Command Prompt and run `composer --version`
- Ensure Composer is installed successfully.



Visual Studio Code

Install VSCode

- Go to the official VS Code website.
- Download and Run the Installer.
- Launch Visual Studio Code.



Unity Hub

Install Unity

- Go to the official Unity website.
- Download the latest version.
- Run the installer and complete setup.
- Open Unity Hub and install the required Unity Editor version.





WARESAFE
SYSTEM

IOT SYSTEM DEVELOPMENT



ESP32 Development Setup

Configure Arduino IDE

- Open Arduino IDE.
- Navigate to File → Preferences
- Add the following URL to Additional Boards Manager URLs:

https://dl.espressif.com/dl/package_esp32_index.json

- Go to Tools → Board → Boards Manager
- Install ESP32 by Espressif Systems.



Required Arduino Libraries

- Install the following libraries using Tools → Manage Libraries:

WiFi.h

MFRC522

Wire

PubSubClient

LiquidCrystal_I2C

SPI

Configure ESP32 Gateway

- Open the ESP32 Gateway source code
- Replace the Wi-Fi credentials and MQTT broker address with your local network configuration before uploading the firmware

```
const char* WIFI_SSID = "Redmi Note 11";  
const char* WIFI_PASS = "1234567890";  
const char* MQTT_SERVER = "broker.emqx.io";  
const int MQTT_PORT = 1883;
```

Upload Program to Board

- Connect ESP32 microcontroller board to the computer
- Open Office Area firmware
- Click Verify then Upload
- Repeat for Warehouse firmware

Note: Set the Serial Monitor to 115200 baud to ensure the output is displayed correctly

MQTT Broker Configuration

Subscribe to all WareSafe topics

Office Area

office/floor1/system
office/floor1/security

Warehouse Area

warehouse/floor2/system
warehouse/floor2/security



Note: Ensure all power supplies are disconnected before assembling the hardware

Hardware Assembly



Step 1. Prepare the Prototype

- Prepare all hardware components for both the Office Area and Warehouse Area

Step 2. Assemble the Sensors

- Connect all sensors and actuators according to their respective wiring diagrams (Schematics Diagram)



Step 3. Verify all wiring connections

- Make sure all sensors are connected correctly and securely



Step 4. Connect the ESP32 to the power supply

- Make sure the firmware has been uploaded to each ESP32 before powering on the system

Step 5. Verify sensor operation

- Trigger each sensor and confirm that sensor status changes correctly, MQTT messages are published and dashboard receives real-time updates

Expected Result: All Area sensors should communicate successfully with the MQTT broker and appear online in the dashboard



WareSafe Prototype Result



Dashboard Monitoring



Schematic Diagram of The Sensor



WARESAFE
SYSTEM

DASHBOARD MONITORING DEVELOPMENT



Setup Laravel Project

Install Dependencies

- Open VS Code and Terminal: Navigate to the WareSafe dashboard-backend project directory.
- Install :

```
composer install  
npm install  
php artisan key:generate
```



Configure Environment File

Configure .env file

- Open file .env

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=waresafe_db  
DB_USERNAME=root  
DB_PASSWORD=
```

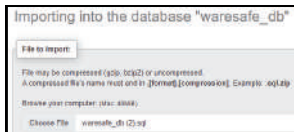
← Configure the system to use mysql

← Update the database name to match the database being used

Setup Database

Create Database

- Access phpMyAdmin through your browser.
- Create a new database named **waresafe_db**
- Select the database and import the provided SQL file.
- Ensure all required tables are created successfully.



Run Database Migration

- Execute `php artisan migrate`
- Verify Migration Completed Successfully.



Dashboard Environment

Run Backend Environment

- Open Command Prompt. Navigate to the project backend directory
- Run `php artisan serve`
- Access the generated URL in a web browser

```
PS C:\Users\...> cd ..\dashboard-backend
PS C:\Users\...> php artisan serve

Laravel server running on [http://127.0.0.1:8000/].

Press Ctrl+C to stop the server
```

Dashboard Monitoring Testing

- Open Command Prompt. Navigate to the project frontend directory
- Run `npm run dev`
- Access the generated URL in a web browser

```
PS C:\Users\...> cd ..\dashboard-frontend
PS C:\Users\...> npm run dev

> dashboard-frontend@0.0.0 dev
> vite

VITE v5.0.3  ready in 346 ms
➜ local: http://localhost:5173/
➜ network: http://192.168.1.100:5173/
```

Dashboard Monitoring Testing

- Login to Dashboard
- Open Monitoring Page
- Trigger Sensor Events
- Verify Real-Time Updates are Displayed





WARESAFE
SYSTEM

CYBER ATTACK INJECTION



Attack Simulation Scenarios

WareSafe provides multiple cyber attack simulation scenarios for cybersecurity awareness and system evaluation. These simulations demonstrate how different cyber threats affect an IIoT monitoring system in a safe and controlled environment without impacting the actual monitoring operation.

Available Simulation Scenarios

Replay Attack Simulation

Simulates the retransmission of previously captured sensor data to demonstrate how repeated messages can create unauthorized access events when system protection is disabled.

Vibration Spoofing Attack Simulation

Simulates falsified vibration sensor data to demonstrate how manipulated readings can trigger false security alarms.

Flooding Attack Simulation

Simulates excessive alarm messages being transmitted to demonstrate how high-volume traffic can disrupt normal monitoring operations.

Security Suppression Attack Simulation

Simulates attempts to suppress or bypass security alerts, demonstrating how attackers may create a false indication that the monitored area is secure.

Defense Mechanism

The defense mechanism protects the IIoT monitoring system by validating incoming data and blocking malicious activities during cyber attack simulations. When defense mode is enabled, the system detects and rejects simulated attacks, allowing only legitimate sensor data to be processed.



WARESAFE
SYSTEM

GAME SIMULATION DEVELOPMENT



Unity Project Setup

Create the Unity Project

- Install Unity Hub and Unity Editor
- Create a new 2D Unity Project named WareSafe
- Open the project through Unity Hub

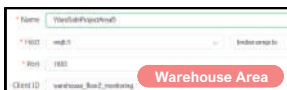
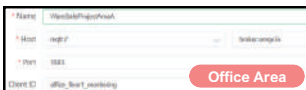


Install Required Packages

- Open Window → Package Manager
- Add package MQTTnet-Unity: <https://github.com/asus4/MQTTnet-Unity.git>
- Add package Ink Unity Integration:
<https://openupm.com/packages/com.inkle.ink-unity-integration/>

Configure MQTT Connection

- Open the MQTT connection script and configure:
Broker IP Address **Client ID** **Port**
- Ensure the MQTT configuration matches the broker used by the WareSafe IoT system.

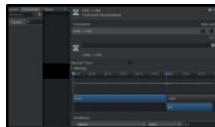


Import Game Assets

- Install aseprite: https://github.com/aseprite/aseprite?utm_source=chatgpt.com
- Import the required project assets, including:
Background Image **UI assets** **Animation sprites** **Character sprites**
- After importing, verify that all assets are available in the Unity Project window

Configure Character Animation

- Create the required character animations, including walk and idle
- Verify that animation transitions operate correctly before testing gameplay.





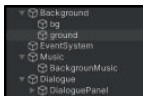
Unity Project Setup

Configure Game Scenes

- Create the required game scenes according to the project design
- Configure scene navigation and transitions between gameplay stages
- Verify that all scenes are connected correctly through Unity Scene Management

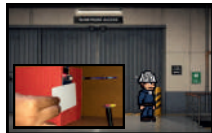
Configure Scene Components

- Configure the required scene objects according to the project requirements
- Add the necessary UI components, interactive objects, background assets, audio resources, and player controller.



Configure Cyber Attack Simulation

- Configure the cyber attack simulation according to the project requirements
- Connect the simulation scripts with the MQTT communication service
- Verify that the simulation interacts correctly with the IoT monitoring system before running the application



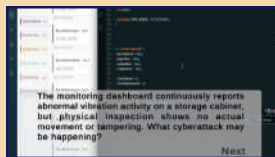
Build and Run the Application

- Open File → Build Settings
- Select the target platform and add the required game scenes to the build
- Click Build or Build and Run to generate the application
- Launch the generated application and verify that it runs successfully

Expected Result: Expected Result: All scenes load successfully and synchronize with the monitoring dashboard



Game Interface Preview



Cybersecurity Learning Quiz

THANK YOU



WARESAFE

IoT-based Security System with Cybersecurity Simulation

Game Development | Cybersecurity | Internet of Things

Joy Adelia Sihombing

001202300077

Zafyra Raisidha Azhari

001202300032

Zelika Hannah

001202300067